Auditable Compositional Question Answering over UK Public Procurement

Anonymous *anonymous@example.com*
University College London

Abstract

The United Kingdom publishes hundreds of thousands of contract award notices, yet the public cannot reliably analyse them in natural language, because the questions that matter are exhaustive aggregations where a fluent wrong answer is worse than none. We introduce, to the best of our knowledge, the first systematic benchmark and executable framework for auditable compositional question answering over UK public procurement. The task ships complete. A convention-explicit knowledge graph of 215,221 contract awards, a benchmark of 12,828 questions each carrying an executable gold program, a typed compositional operator algebra that formalises real procurement analytics, refusal on ambiguous, unsupported and empty questions as scored behaviour, and answer oracles validated by an independent second implementation at 99.88 percent over 14,770 audited answers. Because no annotator pool can label compositional procurement programs at scale, training uses a verified bootstrap. Models propose candidate programs and deterministic checks, compilation, grounding, execution and oracle agreement, decide what may be learned from. An 8-billion-parameter local model trained this way reaches 85.65 percent against 69.76 for the cloud teacher that generated its training data (paired McNemar $p < 10^{-15}$), and 78.31 percent on a sealed challenge set with an independent surface grammar, a constraint-necessity audit, and a 28-point margin over the same teacher. Transplanted to WikiTableQuestions by exchanging only the data-representation layer, the recipe lifts a 22.5 percent zero-shot floor to 44.4 with answer-only supervision and 51.8 with gold programs on the official sealed test, while a four-way attribution audit shows a substantial share of the apparent expressiveness gap to be representation and compilation debt rather than missing reasoning capability. We claim no state of the art. The claim is a task built end to end, and audits strong enough to say precisely what its system can and cannot yet do.

1 Introduction

Public bodies in the United Kingdom publish hundreds of thousands of contract award notices every year. Journalists and auditors ask analytical questions of this record. How much did a council spend on cleaning in 2023. Which supplier received the most awards under a procurement category. Did spending rise after a policy change. An answer to such a question carries the authority of official data, so a fluent but wrong answer is worse than no answer. A system for this task must produce answers that a reader can trace to specific records, and it must refuse when the data cannot support one.

Large language models read these questions well and answer them unverifiably. Nothing in generated text separates a genuine aggregation over all matching records from a plausible guess over a retrieved sample. Retrieval augmentation (Lewis et al., 2020) does not close this gap, because the questions are dominated by exhaustive computation. A count over every contract of every London borough does not fit in a context window. On our development set a retrieval baseline answers 31.5 percent of questions while an untrained 8B model that is forced to emit an executable plan answers 61.5 percent. Forcing an executable plan is worth more than retrieval before any training takes place.

The standard route to a competent small model is distillation from a large one. That route assumes the teacher can be trusted, and on this task it cannot. The strongest cloud pipeline we evaluate answers 69.76 percent

of held-out questions. Imitating it transfers its errors together with its competence. Filtered self-training methods (Zelikman et al., 2022; Gulcehre et al., 2023) replace trust with a pass criterion, but the criteria in use are proxies. A SQL query can execute cleanly and compute the wrong thing. A model can agree with itself and be wrong. Outputs that pass for the wrong reason enter the training pool silently, and none of these methods measures how often that happens.

We introduce, to the best of our knowledge, the first systematic benchmark and executable framework for auditable compositional question answering over UK public procurement data. Procurement knowledge graphs exist, and TheyBuyForYou (Soylu et al., 2022) built one across EU procurement for integration, anomaly detection and search. What has not existed is the task itself in usable form. That means a knowledge graph whose money and identity conventions are explicit enough to verify against, a benchmark whose every question carries an executable gold program, a typed program language that expresses real analytical demands, refusal as a scored behaviour, and an evaluation whose oracles are measured rather than assumed. We build all of it, and we train a system on it without large-scale manual annotation.

The training method follows from the setting. No annotator pool can label tens of thousands of compositional procurement programs, so supervision must come from models, and the task itself supplies a filter stronger than any proxy. Whether a plan compiles, grounds, executes, and matches an independently computed oracle are deterministic checks. We train only on outputs that pass all of them, whichever model produced them. An 8-billion-parameter local model trained this way reaches 85.65 percent on the held-out benchmark of 2,285 questions, against 69.76 percent for the cloud teacher that generated its training data. The gap is significant at $p < 10^{-15}$ under a paired McNemar test (McNemar, 1947) and survives a measured teacher noise floor of 1.1 points.

The recipe transfers. We rebuild only the data layer and move the system to WikiTableQuestions (Pasupat & Liang, 2015), a benchmark of crowd-written questions over web tables that we did not construct. The procurement checkpoint alone moves the zero-shot floor from 22.5 to 27.3 percent. The recipe moves it to 44.4 percent when supervision comes from final answers only, and to 51.8 percent when gold programs exist, both on the official sealed test under a single frozen-configuration run. The checkpoint accounts for 4.8 points of the transfer gain, while verified training accounts for the larger subsequent improvement.

We make five contributions, ordered from the task outward.

First, the task. We define auditable compositional question answering over UK public procurement, from raw releases to a convention-explicit knowledge graph of 215,221 contract awards, with refusal on ambiguous, unsupported and empty questions as first-class behaviour (Sections 3 and 4).

Second, the representation. A typed compositional operator algebra formalises real procurement analytics, set composition, guarded money aggregation, grouped comparison and multi-stage analysis, as closed but recursively compositional executable programs (Section 3).

Third, the evaluation foundation. A program-first benchmark of 12,828 questions with executable gold programs and plan-level split isolation, a sealed challenge set with an independent surface grammar and a constraint-necessity audit, and oracles validated by a second independent implementation at 99.88 percent over 14,770 audited answers (Sections 4 and 5.2).

Fourth, the training method the setting demands. A verified bootstrap in which deterministic execution checks, not teacher trust and not proxy rewards, decide what the model may learn from, requiring no manual program annotation (Section 5.1).

Fifth, the empirical case. The trained local system reaches 85.65 percent at home and 78.31 on the sealed challenge set, 28 points above its cloud teacher there, and the recipe transfers to WikiTableQuestions at the cost of a data layer, where an attribution audit separates representation debt from genuine language limits (Section 5.3).

We claim no state of the art. Recent specialised table systems score higher on WikiTableQuestions than we do. The claim is that a deterministic filter over an executable language turns weak local models into systems that beat their own teachers, across both the home benchmark and an independently constructed challenge set.

2 Related Work

The loop we use, generate candidates, filter them, retrain on the survivors, is standard. We do not claim it. The difference against each neighbour below lies on two axes. The first is what a candidate must prove before it becomes supervision. The second is what happens to candidates that pass wrongly. We state both for every cluster.

Procurement knowledge graphs. TheyBuyForYou (Soylu et al., 2022) integrated cross-national EU procurement into a knowledge graph and built services for data integration, anomaly detection and search. Later work organises procurement documents into navigable knowledge bases. These efforts are relevant because they establish that procurement data rewards graph treatment, and they are not enough for our purpose because none defines question answering as an executable task. They ship no gold programs, no compositional query language, no refusal semantics, and no independently-audited oracles, which are exactly the components a measurable QA task needs.

Filtered self-training. STaR (Zelikman et al., 2022) keeps sampled rationales whose final answer matches a gold label and retrains on them. Rejection-sampling fine-tuning (Yuan et al., 2023) and ReST (Gulcehre et al., 2023) generalise the pattern to reward thresholds and iterated grow-filter-train cycles. ExeSQL (Zhang et al., 2025) bootstraps text-to-SQL with execution success as the filter, and self-correction distillation (Zhu et al., 2026) transfers query generation and error-correction trajectories from a trusted teacher. All of these are relevant because they train small models from filtered model outputs, exactly our setting. The systems reviewed here are not enough for our task, for one measured reason. Their pass criteria are proxies. Execution success accepts queries that run and compute the wrong thing. Agreement accepts consistent errors. Gold-answer matching accepts right answers reached by invalid programs. During our own harvest, 86.7 percent of generated bridge plans passed every deterministic check while only 52.3 percent matched the oracle. A filter blind to that stratum feeds a third of its pool with structurally valid wrong programs and never knows. We retain that stratum with its verifier diagnoses for failure analysis and for controlled negative-training experiments, and we report where it leaks.

Agents over structured knowledge. Pangu (Gu et al., 2023) constrains an LLM to discriminate among logical-form extensions enumerated from the graph. ChatKBQA (Luo et al., 2024a) generates a form and grounds it by retrieval. RoG (Luo et al., 2024b) plans relation paths and reasons over them. StructGPT (Jiang et al., 2023) gives a frozen model iterative read access through fixed interfaces. These systems share our premise that the structure, not the model, is the ground. They are not enough for two reasons. Their training pipelines, as reported, draw no supervision from verifier rejections, so the training value of failure is discarded. And their reported evaluations do not score refusal. Questions that are ambiguous, unsupported by the schema, or matched by no records are first-class citizens of our benchmarks, with three trap families and a faithfulness-gated metric.

Executable table question answering. Classical semantic parsers on WikiTableQuestions reach 37 to 46 percent with weak supervision (Pasupat & Liang, 2015). TaPas (Herzig et al., 2020) and TaBERT (Yin et al., 2020) pre-train table encoders and reach the high forties to low fifties, with table-pretraining successors improving further (Liu et al., 2022; Jiang et al., 2022). Binder (Cheng et al., 2023) and later LLM-with-code systems (Ye et al., 2023; Ni et al., 2023; Wang et al., 2024) pass 65 percent by generating SQL or Python against the table. These works calibrate our transfer numbers, and our answer-only result of 44.4 percent sits inside the classical band while our gold-program result of 51.8 exceeds TaPas-large. They are not enough for our purpose because the program language is chosen for coverage, not for verifiability. Open-ended Python generation does not by itself provide a type checker that a second implementation can replay, a closed grammar that decoding can enforce, or abstention semantics. Our algebra buys those properties and pays for them in coverage, and Section 5.3 measures the price and then decomposes it.

Preference optimisation. Direct preference optimisation (Rafailov et al., 2023) and its variants are the standard way to use negatives. Likelihood displacement work (Razin et al., 2025) shows the objective is most destructive when chosen and rejected responses are similar. Our verifier produces exactly that regime,

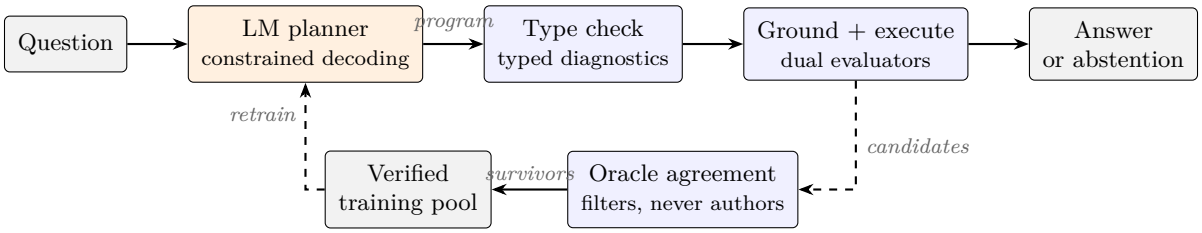

Figure 1: The system. A language model translates the question into a typed program under constrained decoding, and everything downstream is deterministic code. In the bootstrap loop, candidates that compile, ground, execute and match the independently computed oracle become training data. The oracle filters and never authors.

near-miss negatives one slot away from correct plans, and Section 6 reports the resulting hazard as a bounded negative finding rather than a method.

3 The Task and the System

This section specifies the method end to end and, for every load-bearing design decision, states the alternative that was considered, the reason the alternative was rejected, and where possible the measurement that settled the choice. The decisions are not independent. Each one exists to protect the same property, that every answer the system produces can be traced through deterministic code to specific records, and that every training example the system learns from has survived checks no language model can talk its way past. Figure 1 shows the resulting pipeline. A language model appears in one role only, translating a question into a program of a small typed algebra, and everything after that point is deterministic code.

3.1 Data and Knowledge Graph

The raw corpus is five years of UK contract award notices in the Open Contracting Data Standard, 166,277 compiled releases from 2022 to 2026. A field survey conducted before any modelling exposed two properties that shaped everything downstream. First, award-level value fields are empty throughout the corpus while signed values live in contract sub-records, so a naive reader of the documented schema aggregates nothing. Second, 89 percent of buyer references and 86 percent of supplier references use platform account identifiers rather than entity identifiers, and one real organisation can hold dozens of accounts. The Ministry of Defence alone appears under 77 distinct platform identifiers in a single year of data.

The knowledge graph therefore makes conventions explicit rather than implicit. Monetary value is resolved through a fixed fallback order from award to contract to tender value, and every resolved value carries a provenance column and a boolean additivity flag, because tender-level fallback repeats one framework value across several awards and must be excluded from sums. The alternative, treating whichever value field is populated as the value, is what a schema-faithful implementation would do, and it silently double-counts framework agreements whose recorded ceilings reach 51 billion pounds. The additivity flag turns a folklore rule into a guard the executor can check.

Entity resolution runs precision-first through ordered tiers, registry identifier merges first, then safe deterministic variants, then normalised name with region, then name-only merges, with 255 borderline cases adjudicated by a language model into a human-reviewable file and 82,013 singletons left deliberately unmerged. Fuzzy matching is only permitted to produce candidate files for review, never merges. The aggressive alternative, similarity-based merging, was rejected on an asymmetry argument. A false split costs one under-counted organisation and is repairable at query time, which the system does through variant equivalence classes. A false merge silently corrupts every count and sum that touches the merged entity and is not detectable downstream. The result is 204,711 raw aliases resolved to 131,502 canonical organisations, alongside 215,221 contract award nodes.

The graph is stored as typed columnar tables rather than in a graph database. At this scale vectorised joins outperform query-engine round trips, and the choice keeps the executor a deterministic, unit-testable library, which is what later allows a second, independently written implementation to replay every benchmark answer for the oracle audit in Section 4. A graph store would add operational surface without adding expressiveness the workload uses.

3.2 The Operator Algebra

The system answers questions by compiling them into programs of a small typed algebra. Programs are trees over seven value types, and seventeen node types cover filtering with negation, disjunction and computed-set membership, projection, counting, guarded money summation, unique lookup, extremum records, grouping with count or sum metrics, group reduction by extremum or top-k, scalar arithmetic and comparison, set union, intersection and difference, and groupwise combination. The transfer experiment of Section 5.3 adds one generic node, a row-preserving arg-extremum, and nothing else. Appendix A lists the full inventory with type signatures.

The obvious alternative is to let the model generate SQL or Python, which is what the strongest table question answering systems do and what gives them their coverage. The algebra was chosen anyway, for three properties that open-ended generation does not provide and that this task cannot do without. The grammar is closed, so membership is decidable by a recursive type checker whose failures are typed diagnostics rather than stack traces. The grammar is small, so a second evaluator can implement it independently for the oracle audit. And the grammar compiles to a JSON schema, so decoding itself can be constrained to the language. The price is expressiveness, and Section 5.3 measures that price on an external benchmark, at 53 to 55 percent coverage of gold programs before compiler repair, rather than asserting it is small.

3.3 One Stage or Two

The planner is deployed in two configurations, and the choice between them was made by experiment rather than by preference. In the two-stage configuration a reading model first produces a compact understanding brief, eight budgeted sections covering answer type, a closed set of query templates, verbatim question atoms, the answer's dependency chain, and named intermediate entity sets, and a planning model then compiles brief and question into a program. In the one-stage configuration a single model reads the question and emits the program directly.

Both configurations were built because the evidence favoured each in a different regime. On the procurement benchmark the two-stage system is strictly stronger, and the cross-base comparison quantifies why. A fully local system on the weaker base model outperforms the hybrid system on the stronger base by 5.30 points with interval 3.62 to 6.97, meaning briefing quality dominates base capability on this task. Within the second stage, two prompt variants were also measured rather than assumed. A capability-card variant that constrains planning with template shells helps the smaller cloud planner, while the leaner briefing-only variant is better for the stronger planner, whose bridge planning the shells anchor. On the web-table transfer the same comparison reverses. There, a sequence of nine handoff protocols between a reading stage and a planning stage plateaued well below a single fused model that reads and plans in one pass, because the hardest questions require recasting the reading while planning, and any frozen handoff kills the recast. The paper therefore reports the two-stage system where it was measured to win and the fused planner where that was measured to win, and treats the boundary between the regimes as a finding rather than an embarrassment.

3.4 Deterministic Execution and Verification

Everything after planning is deterministic code. A recursive type checker validates the tree, a grounding layer resolves surface strings to canonical entities and slots, and the runtime evaluator executes under the stated conventions, deduplication on the contract identifier, exclusion of empty group keys, exact decimal money arithmetic under the additivity guard, and deterministic tie breaks. A second evaluator, implemented from the raw files with no shared code, exists solely to measure whether the first one is right.

The decision to verify with deterministic checks rather than with a model was forced by two measurements. During benchmark construction a language model was asked to quality-check 9,762 paraphrase rewrites and approved every single one, including the rewrites a mechanical drift detector later rejected, so model checking was demonstrated to be a rubber stamp on this distribution. During harvesting, 86.7 percent of generated bridge plans passed every structural check while only 52.3 percent matched the independent oracle, so structural validity alone admits a large stratum of wrong programs and a filter without an answer oracle cannot see it. Section 5.4 adds a third measurement, a faithful port of a state-of-the-art LLM verifier that contributes exactly zero selection signal on this task even when it is shown the executed answers.

3.5 Decoding as Format Authority

At inference the algebra's grammar is compiled to a JSON schema and enforced token by token through constrained decoding (Kwon et al., 2023). Every evaluated system, including untrained bases, is format-locked. This is a fairness device, and one diagnostic justifies it. Decoded freely, the untrained base emits question-grounded plans in a fluent self-invented schema on 98 of 100 sampled questions and matches the executable contract on 3. The fine-tuned models match the contract on 98 and 99. Fine-tuning therefore teaches conformance to the specific contract, not the ability to plan, and granting the contract to every system through decoding is what makes accuracy differences interpretable as planning quality. Without this control the scoreboard would conflate two skills, and the cheaper one would dominate the measurement.

3.6 Abstention and Gated Repair

Refusal is an output, not a failure. A question may be ambiguous, may name a concept the schema does not carry, or may match no records, and the system abstains through the same typed channel it answers through. An empty result whose literals all trace to the question is treated as the data's answer. A repair loop exists and is gated. It consumes typed diagnostics from failed checks, proposes one revision, and re-enters the same checks, but abstentions and semantically meaningful empty results are final. The gate is not a stylistic preference. An earlier ungated loop converted six correct refusals into confident wrong answers on a development slice, which is the worst possible exchange for a system whose stated purpose is that wrong answers are worse than none.

3.7 The Verified Bootstrap

Training data is manufactured, never annotated. A generator authors a program, executes it under both evaluators, and only then renders a question surface, so every question is born with an executable gold program. A harvest then samples candidate programs from a teacher model or from the student itself at positive temperature, and keeps a candidate only when it compiles, grounds, executes, and matches the oracle answer. Greedy decoding is deliberately avoided during harvest because it can only re-collect what the model already does, and the point of the bootstrap is exploration. The oracle filters and never authors. No gold program or answer is copied into any training text. What fails is retained with its diagnosis for the failure analyses of Section 5.4.

The training ladder runs zero-shot, supervised fine-tuning, rejection sampling fine-tuning on the student's own verified survivors with the teacher absent, and preference optimisation, on two 8B bases under identical configurations. Preference optimisation was evaluated and then abandoned for the primary systems on measured grounds. The verifier's rejects are near-miss negatives one edit from correct plans, which is precisely the regime where likelihood displacement is most destructive, and suppressing them dragged down adjacent correct modes on one base while cleaner preference pools made the effect worse. The deployed fully local systems use the best measured arm per base and additive distillation in place of preference training.

4 Benchmarks and Evaluation Protocol

Plan-first generation. The main benchmark contains 12,828 questions over the knowledge graph, built by instantiating parameterised programs, executing them for oracles, and only then writing question surfaces.

The natural alternative, collecting questions and annotating them with programs afterwards, produces more natural language but cannot guarantee that annotations exist, are executable, or cover the composition space, and its verification cost grows with every question. Plan-first generation makes the gold program a birthright of every question, which matters doubly here because programs are also the training target. Two gates police generation. Gate A re-runs every specification against the full graph and rejects local sampling omissions and multi-answer ambiguity. Gate B has an independent model answer the rendered question from the evidence alone and rejects the question when that answer misses the oracle. Splits separate plans, not surfaces, and three build invariants are enforced mechanically, no plan straddles train and evaluation, identifiers are globally unique, and row counts are conserved across every rebuild.

Quality audit. A comparison against professional benchmark practice exposed six defects in the first build, and each fix is itself testable. Boolean questions answered true 330 times out of 330, so mutated false twins were generated and their oracles recomputed independently. A paraphrase quality check by a language model approved all 9,762 rewrites, so it was replaced by a position-aware mechanical drift detector whose final precision on review was one true defect and zero false positives. Refusal questions were lexically detectable more than half the time, so answerable contrast twins with the cue attribute swapped were added. Gold programs under-specified thresholds and comparison sides that existed only in surface text, so 991 additivity guards, 690 comparison parameters, 316 answer fields and 15 top-k parameters were backfilled. The remaining defect, oracle circularity, is addressed by construction below.

The dual oracle. A second evaluator, implemented from the raw files with no shared code, replays every benchmark answer. Agreement is 99.88 percent over 14,770 audited answers, and all 18 residual disagreements were opened by hand and characterised, none affecting a reported comparison. This is the property the columnar architecture of Section 3.1 was chosen to enable. Correctness of the oracles is a measured quantity in this paper, not an assumption, and every number downstream inherits that property.

PACS. The procurement analytics challenge set exists because a self-built benchmark, however audited, leaves one question open, whether the system merely learned its own generator. PACS answers it with independence that is structural rather than rhetorical. Its specification was frozen before generation, and the evaluated model was frozen before the specification. Seven task families are derived from procurement oversight demands, spending and volume, temporal change, supplier concentration, cross-buyer comparison, overlap and exclusion, relational composition, and disclosure compliance. Program depth and structural exposure are separated into orthogonal labels so that deeper and never-shown are separately attributable, replacing an earlier design that conflated them. Every intent is rendered through three paired surface channels sharing one oracle, an independently authored grammar with no shared stems or connectors as the primary channel, a naturalised rewrite behind deterministic fidelity gates that check operation, negation, comparison direction, quantifier and scope against the gold tree's logical signature, and the training renderer as a diagnostic column only. Isolation is enforced by five zero-overlap gates computed all-pairs before sealing, over question text, gold tree hash, intent and template identity, unseen shape signatures, and near-duplicate surfaces. The sealed test contains 922 rows over 694 intent clusters, the statistical unit is the intent cluster, and confidence intervals use cluster bootstrap. A 93-row human audit preceded unsealing and found one systematic defect, a boolean handling error in the empty-result class that had mislabelled 46 rows, corrected by offline re-execution before any number was read. The frozen system was evaluated on the sealed test exactly once.

WikiTableQuestions. The transfer target is the standard benchmark of 22,033 crowd-written questions over 2,108 web tables (Pasupat & Liang, 2015), evaluated with the official evaluator on the official test split of 4,344 questions. The test set was touched exactly once, under a frozen configuration whose commit hash and file checksums were recorded before launch, and it is treated as consumed thereafter. All development ran on folds of the training split that share no tables with any training pool.

Protocol. Every evaluation is a single model call at temperature zero under constrained decoding, with no repair unless a variant is named explicitly. Answerable questions score by type-aware match against the dual-verified oracle, or by the official evaluator on WikiTableQuestions. Refusal questions score under two

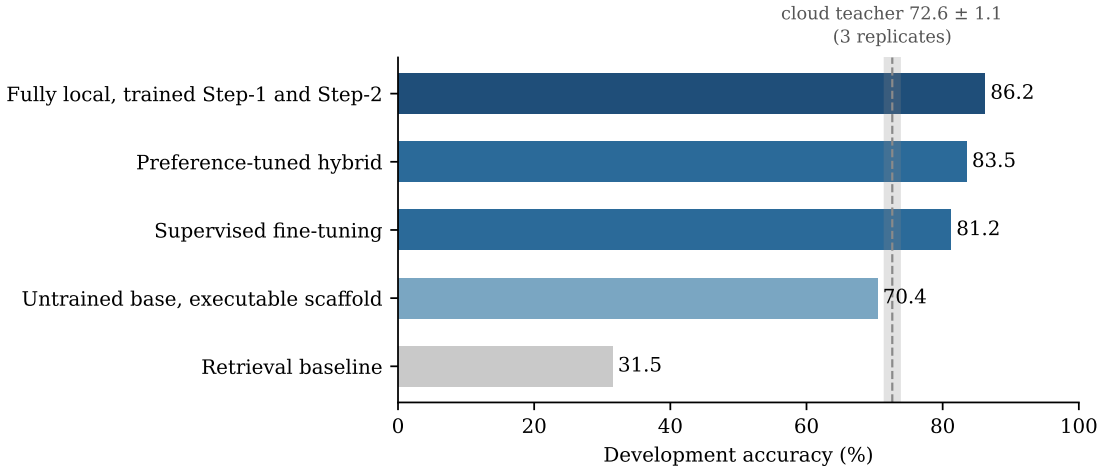

Figure 2: Gain decomposition on the development set. The executable scaffold, not retrieval, sets the floor, and an untrained scaffolded base already touches the teacher's replicate band. Training gains then stack on top of the floor. The shaded band is the cloud teacher's measured variability over three identical runs.

declared metrics, an exact status match and a faithfulness-gated variant in which an empty or multi-valued outcome counts only if every literal of the emitted program traces to the question. Paired systems are compared by exact McNemar tests on discordant pairs (McNemar, 1947). Because the teacher is served by a provider whose decoding cannot be seeded, its variability was measured directly with three identical development runs, giving 72.6 plus or minus 1.1 points, and no single-run teacher delta below three points is claimed anywhere in this paper. Each student configuration is a single frozen training run, so paired significance quantifies test-set disagreement between systems rather than optimisation variance across seeds, and this scoping is stated wherever it applies.

5 Experiments

5.1 Verified Bootstrap on the Home Benchmark

Figure 2 decomposes where the performance comes from, because a single headline number would conflate three different causes. Retrieval-augmented baselines answer 31.5 and 28.8 percent of the development set, and the gap to everything above them is the cost of exhaustive computation that does not fit a context window. An untrained 8B model forced through the executable scaffolding reaches 70.4 percent under the revised pipeline, already within the teacher's measured noise band, so the scaffolding rather than any training sets the floor. Supervised fine-tuning on verified survivors adds 10.8 points, and the fully local system with a trained reading stage reaches 86.2 on development. The format diagnostic of Section 3.5 separates the two remaining explanations for the training gain, contract conformance from planning quality, and attributes the ladder above the floor to planning.

On the held-out test of 2,285 questions the five-system comparison of Figure 3(a) holds with every pairing significant at McNemar $p < 10^{-14}$. The fully local Qwen system reaches 85.65 percent against 69.76 for the cloud teacher that generated its training data, a margin of 15.89 points with 95 percent confidence interval 14.07 to 17.70 and discordant counts of 406 against 43. The result replicates on the second base at 13.57 points, which matters because a single-base result of this kind could be an artefact of one model family's pretraining. The student beats each of the three teacher replicates separately, so the comparison does not rest on provider noise.

Three further analyses report what the headline hides (Figure 4). Moving from development to the 2,285-row test costs the hybrid systems 5.5 and 7.1 points but the fully local systems only 0.5 and 0.9, and the pattern holds on both bases. The interpretation offered is that training on verified survivors removes exactly the

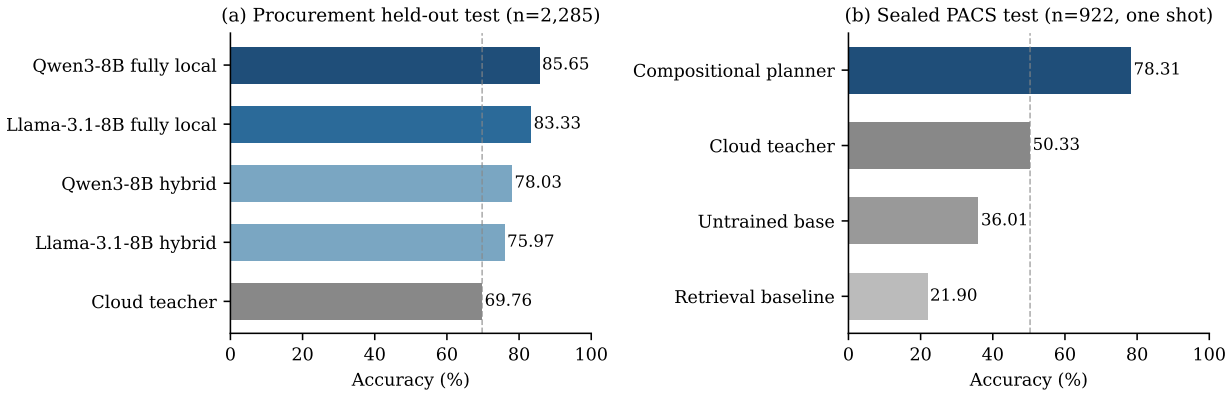

Figure 3: The two scoreboards. (a) On the held-out procurement test both hybrid and fully local students pass the cloud teacher that generated their training data, and the fully local systems pass the hybrids. (b) On the sealed PACS test, evaluated once, the ordering survives with a larger margin. Dashed lines mark the teacher.

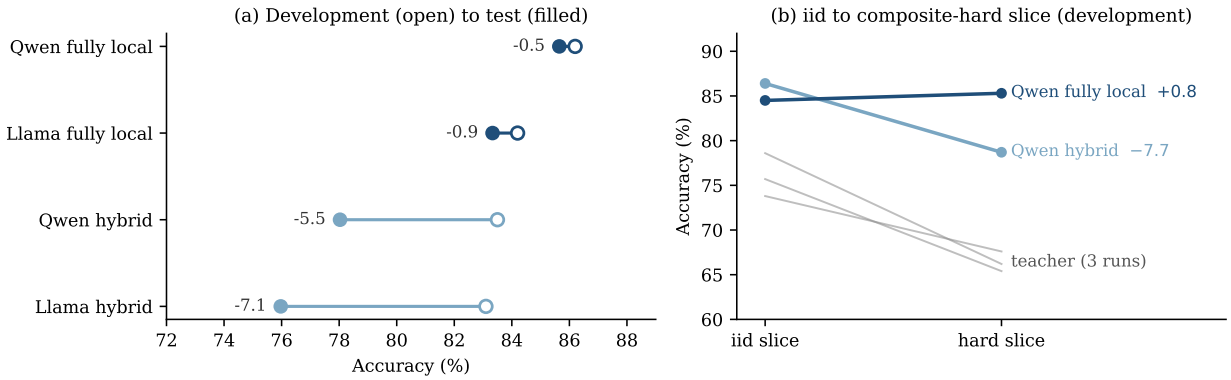

Figure 4: What the headline hides. (a) Development-to-test movement per system. Hybrid systems decay steeply on both bases while fully local systems barely move. (b) On the composite hard slice the teacher and the preference-tuned hybrid fall while the fully local student is flat.

brittleness that surfaces as development-to-test decay, and the cross-base replication is what elevates this from anecdote to evidence. On a composite hard slice of the development set the teacher loses 6 to 12 points and a preference-tuned hybrid loses 7.7 while the fully local student moves up by 0.8, so the student's advantage is largest precisely where questions are hardest, which is the opposite of what a memorisation account would predict. Finally, the bootstrap converges. A second self-harvest round lifts harvest yield from 76.6 to 86.3 percent while the evaluation gain is 0.4 points and not significant, so the verifier-gated loop has a natural ceiling on this task, and the paper reports the ceiling rather than iterating past it.

5.2 A Sealed Benchmark the System Did Not Author

The compositional planner was frozen and evaluated once on the 922 sealed PACS rows. It scores 78.31 percent strict overall and 80.00 on the answerable subset with cluster bootstrap interval 77.06 to 82.92. On the same rows the untrained base scores 36.01, a retrieval baseline 21.90, and the cloud teacher 50.33, so the verified-bootstrap student exceeds its teacher by 27.98 points on questions neither system authored. The comparison is a system-level one between deployable configurations rather than a model comparison under equal decoding, because the teacher's API offers no constrained decoding and 23.1 percent of its rows fail on format alone, and the paper says so rather than presenting the margin as decoding-neutral.

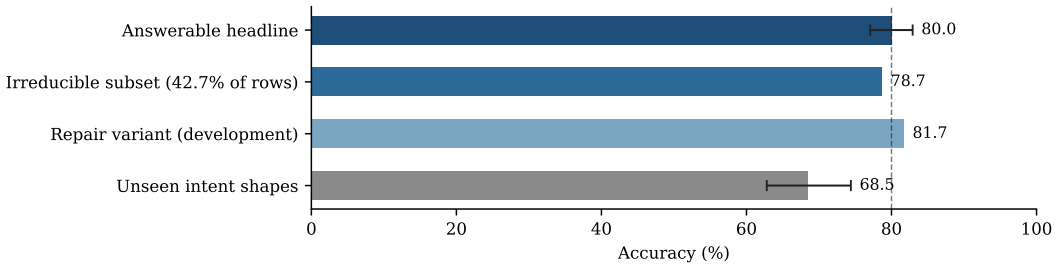

Figure 5: PACS integrity audits around the answerable headline (dashed line, cluster bootstrap interval shown). The planner loses only 1.3 points on the shortcut-resistant irreducible subset, gains 1.7 from a typed-feedback repair variant that is reported but not primary, and pays 11.5 points on intent shapes never demonstrated in training.

Two audits attack the headline from opposite directions (Figure 5). Deleting each oracle-program predicate in turn and re-executing shows that 42.7 percent of answerable rows are irreducible, every predicate load-bearing, and the planner scores 78.7 on that shortcut-resistant subset, within 1.3 points of its answerable headline, so the score is not inflated by questions a partial program could answer by accident. In the other direction a typed-feedback repair loop that acts only on hard failures adds 1.7 points on development rows while leaving every refusal untouched, and the single-call number remains primary. The unseen-exposure axis bounds generalisation honestly. Intent shapes absent from training cost 11.5 points with interval 5.6 to 17.2, composition over demonstrated constructs transfers, constructions never demonstrated remain measurably harder, and the two weakest families, scope binding and universal quantification, are named rather than averaged away.

5.3 Transfer to WikiTableQuestions

The transfer is a controlled experiment on portability, and its control is strict. The algebra, the type checker, and both evaluators moved unchanged. What was rebuilt is exactly the data representation layer, a loader that exposes typed views over dirty cells, numeric twins, year and date extractions, composite-part splits, and row order as data, a column-aware value linker that grounds question tokens to real cells, and the one new operator. Each addition was admitted by measurement rather than by need. The linker faced a four-arm ablation in which real cell candidates gain 4.0 points over a schema-only baseline while random cells lose 1.7, so its value is grounding rather than added context, and the row-preserving arg-extremum was added only after a construct census of development failures showed row-order questions were a leading class.

On the official test set, evaluated once under the recorded manifest, the four rungs of Figure 6(a) are each significant at $p < 5 \times 10^{-18}$. The untrained base scores 22.51, the procurement checkpoint 27.33, answer-only bootstrapping 44.43, and gold-program training 51.80. The checkpoint therefore carries a 4.8-point zero-shot prior across domains, and verified training carries 17 to 24.5 points beyond it. The answer-only figure matches classical weakly supervised parsers, the gold-program figure exceeds early table-pretrained baselines such as TaPas (Herzig et al., 2020), and neither approaches recent specialised systems (Cheng et al., 2023; Ye et al., 2023; Wang et al., 2024). The remainder of this subsection decomposes that gap instead of excusing it.

A differential audit against 11,276 gold SQL programs (Shi et al., 2020) separates where the gap lives, stage by conditional stage (Table 1). The algebra expresses 53 to 55 percent of gold programs. Translated programs match the reference denotation 92 to 93 percent of the time, and executed in-coverage programs recover the target answer in 94.1 percent of cases, so performance is coverage-limited rather than execution-limited. The gold SQL annotations themselves reproduce the benchmark answer in only 88.9 percent of cases, so the annotation layer is treated as a noisy reference rather than an absolute oracle. A closure pass that lowers SQL constructs onto existing nodes and adds typed loader views raises coverage to 61.0 percent with zero regressions, and its attribution matrix is exact, 562 questions recovered by the translator alone, 154 by loader

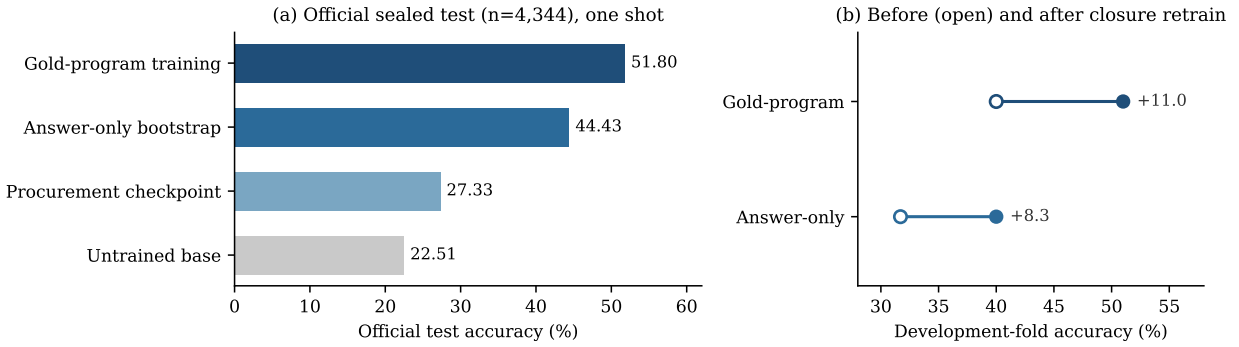

Figure 6: WikiTableQuestions transfer. (a) The four rungs on the official sealed test, touched once. (b) Retraining after representation-and-compilation closure lifts the development fold in both supervision regimes.

Table 1: Where the WikiTableQuestions gap lives. Stage-by-stage decomposition against 11,276 gold SQL programs. Each rate conditions on the stage above it.

Stage	Rate
Gold programs expressible in the algebra (before closure)	53–55%
after translator and loader closure, zero regressions	61.0%
Translated programs matching the reference denotation	92–93%
Executed in-coverage programs recovering the target answer	94.1%
Gold SQL annotations reproducing the benchmark answer	88.9%

views alone, 14 jointly, and 4,466 remaining outside the language (Figure 7). One seventh of the apparent expressiveness gap was therefore compiler and representation debt rather than a limit of the language design.

The decisive test of the coverage account is interventional rather than correlational. If capability is bounded by coverage, then widening coverage and retraining should move accuracy in both supervision regimes, and it does. Retraining after closure lifts the gold-program model from 40.0 to 51.0 on the development fold and the answer-only model from 31.7 to 40.0 (Figure 8). Both slopes exceed the coverage increment itself, consistent with in-coverage questions also becoming easier to learn once the representation stops fighting the planner.

5.4 Design Validations

The choices of Section 3 rest on measurements, and this subsection reports the three that carry the most weight. All of them run on development data.

What fine-tuning actually teaches. The schema-validity diagnostic decodes the untrained base without constraints on 100 development questions. It emits question-grounded plans with real entities on 98, and matches the executable contract on 3. The tuned models match the contract on 98 and 99. Ten randomly selected base transcripts were read by hand to confirm the grounded-but-off-schema pattern rather than trusting the gate, and the diagnostic itself survived three prior gate bugs of opposite sign before its numbers were accepted. The conclusion, that tuning teaches contract conformance while constrained decoding grants the contract to everyone, is what licenses reading the accuracy ladder as planning quality.

Model verification versus deterministic verification. A recent framework replaces discrete judge scores with expectations over scoring token distributions and reports state-of-the-art verification across agentic benchmarks (Kwok et al., 2026). Its applicability here was tested directly, as a selector among four sampled candidates per question that all pass the deterministic gate, where a perfect selector would gain about five points over taking the first survivor. Two ports were evaluated, a minimal one that scores immediately and a faithful one with the framework's letter scale, pairwise comparisons, free-form analysis before scoring, and the

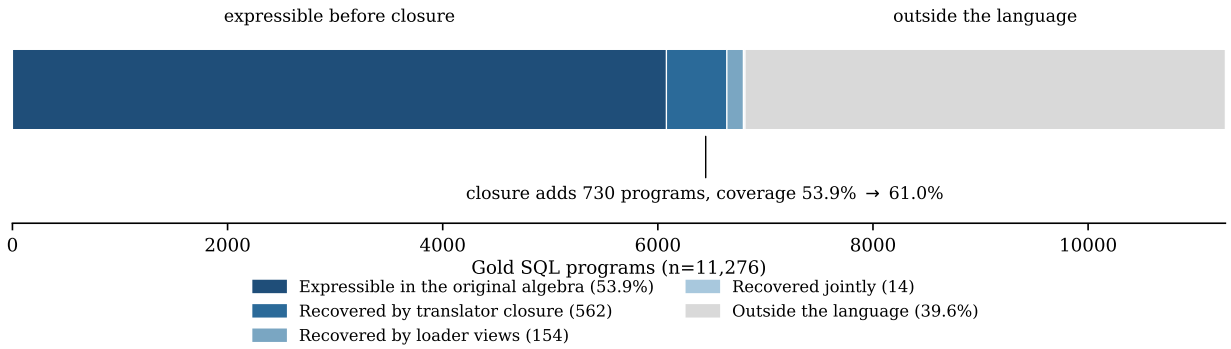

Figure 7: Attribution over the 11,276 gold SQL programs. The closure pass recovers 562 questions through the translator alone, 154 through typed loader views alone and 14 jointly, moving coverage from 53.9 to 61.0 percent. The remaining 4,466 sit outside the language, and the two largest remaining families are specified as typed extensions and deliberately not built mid-evaluation.

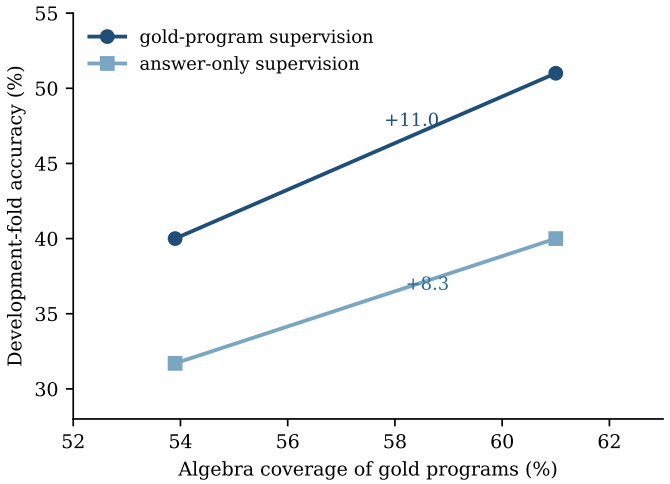

Figure 8: Capability tracks coverage. Widening algebra coverage from 53.9 to 61.0 percent through compiler and representation closure, then retraining, lifts the development fold under both supervision regimes.

executed answers visible in context. The faithful port selects no better than position, a net contribution of exactly zero, and the minimal port selects slightly worse than position. A deterministic reflector-style scorer over execution-derived fault features, zero-row filter probes, empty answers, answer-kind mismatches with the question word, question-literal echoes, recovers 2.0 of the available 5.0 points with paired discordants of seven gains against one loss (Figure 9). Since the faithful port saw the same executed answers the reflector saw, the gap is one of inductive bias rather than information access. On this task the verification signal lives in execution, and judgement adds nothing that rules do not already capture. This is the third independent measurement behind the paper's deterministic-first design, and it also bounds the practical value of best-of-k selection itself, since greedy decoding already sits within two points of the sampling oracle.

Near-miss negatives and preference training. The verifier's rejects are near-miss negatives, typically one slot from a correct plan. Preference optimisation over such pairs helped one base on development and hurt the other, and cleaning the preference pool made the damage worse, which is the signature of likelihood displacement rather than of noisy data (Rafailov et al., 2023; Razin et al., 2025). The deployed systems

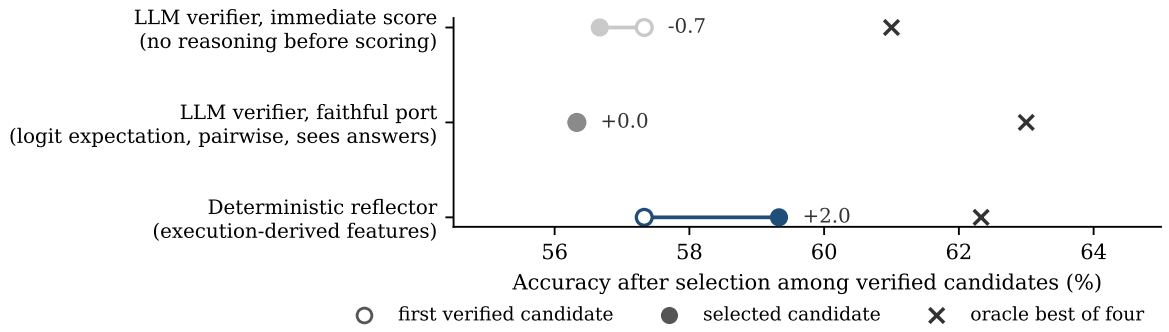

Figure 9: Three selectors among verified candidates on the development fold, each shown against its own first-candidate baseline (open marker) and the best-of-four oracle (cross). The deterministic reflector over execution-derived features is the only selector that beats position, and the faithful LLM verifier port contributes exactly zero despite seeing the executed answers.

therefore use additive distillation, and the negative result is reported as a boundary of the method rather than buried.

6 Discussion and Limitations

The primary claim is about a task and the system that solves it, and the teacher comparison inside that claim is scoped carefully. The teacher operates zero-shot while the students are tuned in domain, so the scoreboard shows that a verified bootstrap beats deploying the teacher it was distilled from, not that an 8B model outranks its teacher in general ability. PACS, where both sides face questions neither authored, shows the gap survives on independent surfaces with a larger margin.

The scale of the evidence supports deployment-level claims rather than prototype-level ones. The system operates over the full five-year national corpus, 215,221 award records and 131,502 organisations, answers a 12,828-question benchmark whose oracles are verified at 99.88 percent by an independent implementation, and was evaluated once on each of two sealed test sets it could not have seen. The practical impact is a local, auditable analyst for public procurement that runs on a single GPU, answers exhaustive aggregation questions that retrieval cannot, refuses when the data cannot support an answer, and produces programs a reviewer can execute rather than prose a reviewer must trust.

Refusal did not transfer free. On a paired legacy comparison the compositional planner wins the answerable subset by 4.4 points yet loses explicit abstention badly, because its training mix carried only 37 refusal demonstrations. Under the pre-registered bar of no key-bucket regression, no retirement claim is made for the older system. Competence follows demonstrations, and refusal is a competence.

The transfer numbers are coverage-limited, and the limitation is measured rather than estimated. The algebra expresses roughly half of gold programs, the planner reaches 85.3 percent of what the expressible subset can express, and the closure audit shows a seventh of the missing half was compiler debt rather than language limit. The two largest remaining gaps, general expression predicates and ordered-relation navigation, are specified as typed extensions and deliberately not built, because widening a language mid-evaluation would dissolve the boundary that makes the measurements interpretable.

Four further limits bound the claims. The home benchmark is self-built, with its integrity measured rather than assumed, and its question surfaces are model-generated under style controls, which is not user language. The WikiTableQuestions test set is consumed, so future work on that benchmark reports development folds or a new holdout. Probe evidence of composition beyond demonstrated constructs is bounded at 11.5 points of unseen-exposure cost. And the evidence remains confined to executable structured-data analytics over one procurement graph and one table benchmark, so claims about other domains are conjectures this paper does not make.

7 Conclusion

UK public procurement is public data that the public cannot reliably analyse in natural language. This work defined that task end to end, a convention-explicit knowledge graph, a program-first benchmark with refusal as scored behaviour, a typed compositional program language, and oracles whose correctness is measured by an independent second implementation. On that foundation a verified bootstrap, deterministic checks deciding what a model may learn from, trains an 8B local system to 85.65 percent, past the cloud teacher that generated its training data, and to 78.31 on a sealed challenge set neither system authored. Rebuilt on web tables at the cost of a data layer, the same recipe turns a 22.5 percent floor into 44.4 without a single gold program and 51.8 with them. Language coverage is now the dominant measured bottleneck, program discovery remains incomplete, and the audits in this paper are what make that distinction measurable.

References

Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. Binding language models in symbolic languages. In *International Conference on Learning Representations (ICLR)*, 2023.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yu Gu, Xiang Deng, and Yu Su. Don't generate, discriminate: A proposal for grounding language models to real-world environments. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Zhengbao Jiang, Yi Mao, Pengcheng He, Graham Neubig, and Weizhu Chen. OmniTab: Pretraining with natural and synthetic data for few-shot table-based question answering. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2022.

Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. LLM-as-a-Verifier: A general-purpose verification framework. *arXiv preprint arXiv:2607.05391*, 2026.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. TAPEX: Table pre-training via learning a neural SQL executor. In *International Conference on Learning Representations (ICLR)*, 2022.

Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, and Anh Tuan Luu. ChatKBQA: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. In *Findings of the Association for Computational Linguistics: ACL*, 2024a.

Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations (ICLR)*, 2024b.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning (ICML)*, 2023.

Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Noam Razin, Sadhika Malladi, Adithya Bhaskar, Danqi Chen, Sanjeev Arora, and Boris Hanin. Unintentional unalignment: Likelihood displacement in direct preference optimization. In *International Conference on Learning Representations (ICLR)*, 2025.

Tianze Shi, Chen Zhao, Jordan Boyd-Graber, Hal Daumé III, and Lillian Lee. On the potential of lexico-logical alignments for semantic parsing to SQL queries. In *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

Ahmet Soylu, Oscar Corcho, Brian Elvesæter, Carlos Badenes-Olmedo, Tom Blount, Francisco Yedro Martínez, Matej Kovacic, Matej Posinkovic, Ian Makgill, Chris Taggart, Elena Simperl, Till C. Lech, and Dumitru Roman. TheyBuyForYou platform and knowledge graph: Expanding horizons in public procurement with open linked data. *Semantic Web*, 13(2):265–291, 2022.

Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *International Conference on Learning Representations (ICLR)*, 2024.

Yunhu Ye, Binyuan Hui, Min Yang, Binhua Li, Fei Huang, and Yongbin Li. Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. TaBERT: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Jipeng Zhang, Haolin Yang, Kehao Miao, Ruiyuan Zhang, Renjie Pi, Jiahui Gao, and Xiaofang Zhou. ExeSQL: Self-taught text-to-SQL models with execution-driven bootstrapping for SQL dialects. In *Findings of the Association for Computational Linguistics: EMNLP*, 2025.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2024.

Yushan Zhu, Wen Zhang, Long Jin, Mengshu Sun, Ling Zhong, Zhiqiang Liu, Juan Li, Lei Liang, Chong Long, Chao Deng, and Junlan Feng. Self-correction distillation for structured data question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

A Operator Algebra Reference

Programs are trees over seven value types. RECORDS is a subset of the flat record universe, VALUES a distinct set of scalars, GROUPS a mapping from group key to number, RANKING an ordered list of key and number pairs, and NUMBER, VALUE and BOOL are scalars. Table 2 lists the closed node inventory with type signatures. Depth is capped at 16 and node count at 64. Every check failure raises a typed diagnostic with a path into the tree, which is what the gated repair loop consumes.

Table 2: Node inventory. The seventeen procurement nodes plus the one node added for the WikiTableQuestions transfer (marked †). Predicates inside `filter` support base constraints (`eq`, `in`, `contains`, `exists`, `gte`, `lte`), negation, disjunction, and computed-set membership (`in_expr`, a semijoin or antijoin on a subtree).

Node	Signature	Meaning
`filter`	preds → RECORDS	subset by predicate conjunction
`values`	RECORDS → VALUES	distinct field values
`count`	RECORDS → NUMBER	record count
`size`	VALUES → NUMBER	set cardinality
`sum`	RECORDS → NUMBER	guarded money summation
`exists`	RECORDS → BOOL	non-emptiness
`select`	RECORDS → VALUE	unique field lookup
`extreme`	RECORDS → VALUE	argmax or argmin record
`groupby`	RECORDS → GROUPS	grouped count or sum
`argext`	GROUPS → VALUE	extremum group key
`top`	GROUPS → RANKING	top-k groups
`num`	literal → NUMBER	numeric literal
`combine`	NUMBER^2 → NUMBER/BOOL	arithmetic or comparison
`vcompare`	VALUE → BOOL	scalar comparison, date-aware
`setop`	VALUES^2 → VALUES	union, intersection, difference
`gcombine`	GROUPS^2 → GROUPS	groupwise combination
`keys_where`	GROUPS → VALUES	keys passing a threshold
`extreme_rows`†	RECORDS → RECORDS	row-preserving arg-extremum

B Audit Protocols and Supplementary Results

Dual-oracle audit. The independent evaluator was implemented from the raw releases with no shared code and replays every benchmark answer. Agreement is 99.88 percent over 14,770 audited answers. All 18 residual disagreements were opened by hand and characterised, and none affects a reported comparison.

PACS isolation gates. Five zero-overlap gates hold between training and sealed test at the level of question text, logical signature, surface template, entity anchors, and program hash. A 93-row human audit preceded unsealing and found one systematic defect, a boolean handling error in the empty-result class affecting 46 rows, corrected by offline re-execution before any number was read.

Constraint-necessity audit. Each predicate of each oracle program is deleted in turn and the program re-executed. A row is irreducible when every deletion changes the answer. 42.7 percent of answerable sealed rows are irreducible, and the planner scores 78.7 on that subset against 80.00 on the full answerable set.

Trap-cue disclosure. Nineteen of 2,285 test rows string-match cue phrases derived from development refusal traps. Excluding them moves every headline by at most 0.21 points and changes no ranking and no significance verdict.

Method ladder. Table 3 reports the development-set ladder over training methods and bases under the revised pipeline, and the harvest yields that feed them. Preference optimisation helps one base and hurts the other, consistent with the near-miss hazard of Section 5.4, and the deployed fully local systems use the best arm per base. Self-harvest yield exceeds teacher harvest yield on both bases, and a second self-harvest round raises yield further, 76.6 to 86.3 percent on answerable questions, while adding only 0.4 evaluation points, not significant, which is where the loop stops.

Table 3: Development ladder (n=260) and harvest yields. Yields are data-engine acceptance rates on answerable questions, a different metric from system accuracy, and are never compared against it.

Arm	Qwen3-8B	Llama-3.1-8B
Zero-shot scaffolded	70.4	60.0
Supervised fine-tuning	81.2	83.1
Rejection-sampling fine-tuning	81.5	82.7
Preference optimisation	83.5	77.3
Fully local (best arm)	86.2	84.2
Teacher harvest yield	65.5	
Self-harvest yield, round 1	76.6	78.1
Self-harvest yield, round 2	86.3	–

C Reproduction Details

Students are Qwen3-8B (Qwen Team, 2025) and Llama-3.1-8B, adapted with rank-64 LoRA (Hu et al., 2022) under 4-bit QLoRA quantisation (Dettmers et al., 2023) in LLaMA-Factory (Zheng et al., 2024), with per-base configurations identical apart from base and chat template. Serving uses vLLM (Kwon et al., 2023) with guided JSON decoding compiled from the algebra grammar, and every local rung including the untrained zero-shot baseline is served through the same guided path. Harvest sampling uses positive temperature, since greedy sampling cannot explore beyond what the model already solves, and evaluation uses temperature zero with a single call per question. Training and evaluation ran on single 80GB A100 and 96GB H100 devices. The WikiTableQuestions official run was executed once under a recorded manifest of commit hash, file checksums, decoding parameters and adapter identity, and the test set is treated as consumed. The experiment ledger records every experiment behind this paper, including the negative results reported in Sections 5.4 and 6, under a fixed rule that any rate-type number is entered only after ten deterministically sampled raw transcripts, passes and failures both, have been read by a human. Full training configurations, prompts, manifests and the ledger ship with the code.